Documentation mise en place d’une base de données SQL

Lauret Chauncey

BTS SIO

Sommaire

I-Modèle relationnel 3

II-Script Base de la donnée 3

II.1 – Création de la base de données 3

II.2 – Création des tables + insertion 3

II.3 – Contraintes 7

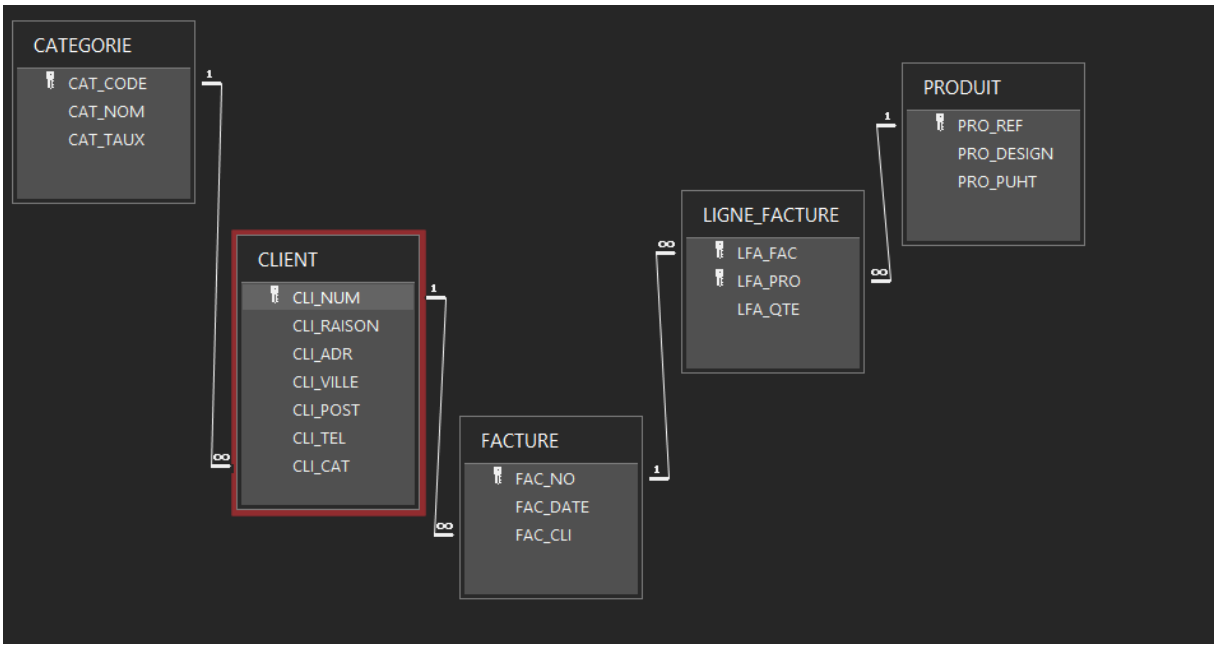
III-Création des utilisateurs 8

IV-Résultat 8

V-Test requêtes SQL 9

VI-Annexe 16

I- Modèle relationnel



Ce modèle va nous permettre de liés toutes les tables entre elles.

II- Script Base de la donnée

II.1 – Création de la base de données

```
create database `Maison_Pain_d'Epices`;

use `Maison_Pain_d'Epices`;
```

II.2 – Création des tables + insertion

Nous allons créer des tables et les insertions grâce a un éditeur de code. Nous utiliserons les lignes de code suivante pour le faire :

- Table catégorie :

```sql
-- Structure de la table `categorie`
--

CREATE TABLE IF NOT EXISTS `categorie` (
  `cat_code` varchar(50) NOT NULL,
  `cat_nom` varchar(50) NOT NULL,
  `cat_taux` varchar(50) NOT NULL,
  PRIMARY KEY (`cat_code`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```sql
--
-- Contenu de la table `categorie`
--

INSERT INTO `categorie` (`cat_code`, `cat_nom`, `cat_taux`) VALUES
('BOU', 'Boulangerie', '5,50%'),
('EPI', 'Epicerie', '0%'),
('HYP', 'Hypermarché', '12.50%'),
('PAT', 'Patisserie', '7%'),
('SUP', 'Supermarché', '10%');
```

- Table client :

```sql
--
-- Structure de la table `client`
--

CREATE TABLE IF NOT EXISTS `client` (
  `cli_num` INT NOT NULL,
  `cli_raison` varchar(50) NOT NULL,
  `cli_adr` varchar(50) NOT NULL,
  `cli_ville` varchar(50) NOT NULL,
  `cli_post` varchar(50) NOT NULL,
  `cli_tel` varchar(50) NOT NULL,
  `cli_cat` varchar(50) NOT NULL,
  PRIMARY KEY (`cli_num`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```sql
--
-- Contenu de la table `client`
--

INSERT INTO `client` (`cli_num`, `cli_raison`, `cli_adr`, `cli_ville`, `cli_post`, `cli_tel`, `cli_cat`) VALUES
(1, 'Anis et Miel', '7 Chemin de Cassis' , 'Marseille', '13010', '04 91 45 13 12', 'BOU'),
(2, 'Auchan', 'Zone industirel', 'Aubagne', '13400', '04 91 26 45 23', 'HYP'),
(3, 'Casino', 'La Valentina', 'Marseille', '13400', '04 91 85 54 42', 'HYP'),
(4, 'Leclerc', 'Plan de campagne', 'Marseille', '13601', '04 91 25 54 47', 'HYP'),
(5, 'Intermarché', '30 Rue des ponts', 'Mallemort', '13456', '04 42 45 48 87', 'SUP'),
(6, 'Casino', 'Bd Paul Claudel', 'Marseille', '13010', '04 91 45 23 87', 'SUP'),
(7, 'Les moulins de provence', '74 Bd Romain Rolland','Marseille', '13010', '04 97 45 52 23', 'BOU'),
(8, 'Les moulins de provence', '45 Av Marcel Pagnol', 'Aubagne', '13400', '04 91 25 48 78', 'PAT');
```

- Table facture :

```sql
--
-- Structure de la table `facture`
--

CREATE TABLE IF NOT EXISTS `facture` (
  `fac_no` INT NOT NULL,
  `fac_date` DATE NOT NULL,
  `fac_cli` INT NOT NULL,
  PRIMARY KEY (`fac_no`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```sql
--
-- Contenu de la table `facture`
--

INSERT INTO `facture` (`fac_no`, `fac_date`,`fac_cli`) VALUES
(1, '2010-09-08',1),
(2, '2010-10-02',2),
(3, '2010-10-10',2),
(4, '2010-10-20',4),
(5, '10-11-22',3),
(6, '2010-11-24',3),
(7, '2010-02-01',7),
(8, '2010-02-03',5),
(9, '2010-05-08',4);
```

- Table ligne_facture

```sql
--
-- Structure de la table `ligne_facture`
--

CREATE TABLE IF NOT EXISTS `ligne_facture` (
  `lfa_fac` int NOT NULL,
  `lfa_pro` varchar(50) NOT NULL,
  `lfa_qte` int NOT NULL,
  PRIMARY KEY (`lfa_fac`,`lfa_pro`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```sql
--
-- Contenu de la table `LIGNE_FACTURE`
--

INSERT INTO `ligne_facture` (`lfa_fac`, `lfa_pro`, `lfa_qte`) VALUES
(1, 'F001', '100'),
(1, 'L001', '10'),
(2, 'L001', '75'),
(2, 'L002', '50'),
(2, 'S001', '150'),
(3, 'F001', '125'),
(3, 'L001', '10'),
(3, 'S002', '30'),
(4, 'L002', '89'),
(4, 'L003', '55'),
(5, 'L001', '74'),
(5, 'S001', '44'),
(6, 'F001', '15'),
(6, 'L001', '10'),
(6, 'S001', '4'),
(7, 'L002', '10'),
(7, 'S002', '5'),
(8, 'L002', '10'),
(8, 'L003', '50'),
(8, 'S001', '20'),
(9, 'L001', '4'),
(9, 'S002', '20');
```

- Table produit :

```sql
--
-- Structure de la table `produit`
--

CREATE TABLE IF NOT EXISTS `produit` (
  `pro_ref` varchar(50) NOT NULL,
  `pro_design` varchar(50) NOT NULL,
  `pro_puht` int NOT NULL,
  PRIMARY KEY (`pro_ref`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

```sql
--
-- Contenu de la table `produit`
--

INSERT INTO `produit` (`pro_ref`, `pro_design`, `pro_puht`) VALUES
('F001', 'Farine de blé', '2.40€' ),
('F002', 'Farine de seigle', '3.55€' ),
('L001', 'Levure de bière', '21.50€' ),
('L002', 'Levain naturel', '11.45€' ),
('L003', 'Levure chimique', '10.50€' ),
('S001', 'Sucre glacé', '11.00€' ),
('S002', 'Sucre vanillé', '8.30€' );
```

II.3 – Contraintes

Une fois toutes les tables créer nous allons les liées entre elles grâce au modèle relationnel. Pour cela on va mettre en place des clés primaires et des clés étrangères.

```sql
--
-- Contraintes pour les tables exportées
--

--
-- Contraintes pour la table `client`
--
ALTER TABLE `client`
  ADD CONSTRAINT `cli_num_fk` FOREIGN KEY (`cli_cat`) REFERENCES `categorie` (`cat_code`);

--
-- Contraintes pour la table `facture`
--
ALTER TABLE `facture`
  ADD CONSTRAINT `fac_no_fk` FOREIGN KEY (`fac_cli`) REFERENCES `client` (`cli_num`);


--
-- Contraintes pour la table `ligne_facture`
--
ALTER TABLE `ligne_facture`
  ADD CONSTRAINT `lfa_fac_fk` FOREIGN KEY (`lfa_fac`) REFERENCES `facture` (`fac_no`),
  ADD CONSTRAINT `lfa_pro_fk` FOREIGN KEY (`lfa_pro`) REFERENCES `produit` (`pro_ref`);
```

III- Création des utilisateurs

Afin que les différents utilisateurs puissent se connecter à la base de données nous allons créer des utilisateurs qui ont des privilèges différents en fonction de leur poste dans l'entreprise.

- Admin :
  L'admin aura tous les droits sur la base de données, on peut voir ça grâce au GRANT ALL PRIVILEGES

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY '5R5sLq1u/$Rl3:';
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;
```

- User :
  L'user pourra uniquement lire les données de la base de données, grâce au GRANT SELECT

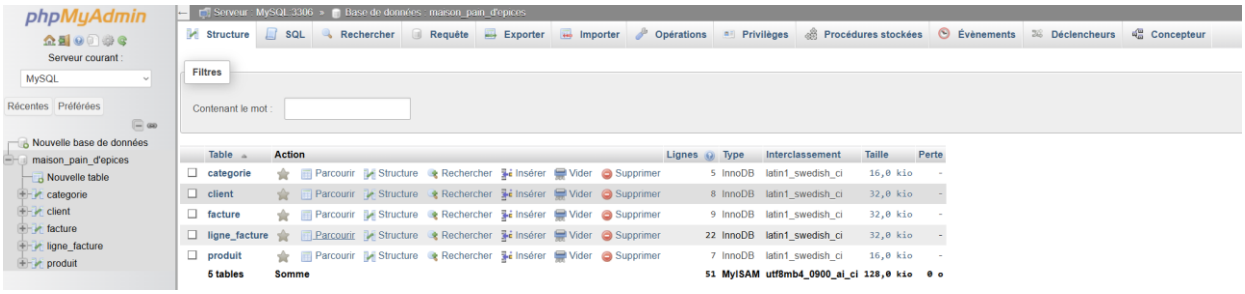
```
CREATE USER 'user' IDENTIFIED BY 'user';
GRANT SELECT ON  *.* TO 'user';
```

- Gestion :
  L'utilisateur Gestion pourra lire est modifier les données de la base de données, on voit ça grâce au GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, FILE, INDEX, ALTER, CREATE TEMPORARY TABLES, CREATE VIEW, EVENT, TRIGGER, SHOW VIEW, CREATE ROUTINE, ALTER ROUTINE, EXECUTE ON *.* TO 'gestion'

```
CREATE USER 'gestion' identified by 'xUk;3XoZh4*2#3';
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, FILE, INDEX, ALTER, CREATE TEMPORARY TABLES, CREATE VIEW, EVENT, TRIGGER, SHOW VIEW, CREATE ROUTINE, ALTER ROUTINE, EXECUTE ON *.* TO 'gestion' ;
```

IV- Résultat

Une fois que les tables et les utilisateurs sont créés on peut aller dans phpMyAdmin afin de visualiser les tables et les utilisateurs de la base de données



V- Test requêtes SQL

Pour vérifier que la base de données fonctionne bien nous allons effectuer quelques tests :

Combien y-a-t-il de catégorie ?

SELECT COUNT(*) FROM `Categorie`;

COUNT(*)
5

Combien y-a-t-il de client ?

SELECT COUNT(*) FROM `Client`;

COUNT(*)
8

Combien y-a-t-il de produit ?

SELECT COUNT(*) FROM `Produit`;

COUNT(*)
7

Quel est le produit le plus cher ?

SELECT `PRO_DESIGN`, `PRO_PUHT` FROM `Produit` WHERE `PRO_PUHT` = (SELECT MAX(`PRO_PUHT`) FROM `Produit`);

←T→				PRO_DESIGN	PRO_PUHT
☐	Éditer	Copier	Supprimer	Levure de bière	22

Quel est le produit le moins cher ?

SELECT `PRO_DESIGN`, `PRO_PUHT` FROM `Produit` WHERE `PRO_PUHT` = (SELECT MIN(`PRO_PUHT`) FROM `Produit`);

←T→				PRO_DESIGN	PRO_PUHT
☐	Éditer	Copier	Supprimer	Farine de blé	2

Quel est le prix moyen des produits ?

SELECT AVG(`PRO_PUHT`) AS `Prix_Moyen` FROM `Produit`;

Prix_Moyen
9.8571

Combien de factures ont fait les clients ? (Résultat regroupé par numéro de client)

SELECT `FAC_CLI`, COUNT(`FAC_NO`) AS `Nombre_de_Factures` FROM `Facture` GROUP BY `FAC_CLI`;

FAC_CLI	Nombre_de_Factures
1	1
2	2
3	2
4	2
5	1
7	1

Combien de produits différents ont été vendus par facture ?

SELECT `LFA_FAC`, COUNT(DISTINCT `LFA_PRO`) AS `Nb_Produits_Diff` FROM `Ligne_Facture` GROUP BY `LFA_FAC`;

LFA_FAC	Nb_Produits_Diff
1	2
2	3
3	3
4	2
5	2
6	3
7	2
8	3
9	2

Combien de quantité (tout produit confondu) ont été vendues par facture ?

SELECT `LFA_FAC`, SUM(`LFA_QTE`) AS `Total_Quantite` FROM`Ligne_Facture` GROUP BY `LFA_FAC`;

		LFA_FAC	Total_Quantite
Copier	Supprimer	1	110
Copier	Supprimer	2	275
Copier	Supprimer	3	165
Copier	Supprimer	4	144
Copier	Supprimer	5	118
Copier	Supprimer	6	29
Copier	Supprimer	7	15
Copier	Supprimer	8	80
Copier	Supprimer	9	24

Quelle est par facture la plus grande quantité vendue ?

SELECT `LFA_FAC`, MAX(`LFA_QTE`) AS `Max_Quantite` FROM`Ligne_Facture` GROUP BY `LFA_FAC`;

LFA_FAC	Max_Quantite
1	100
2	150
3	125
4	89
5	74
6	15
7	10
8	50
9	20

Quelle quantité au total a été vendu chaque produit, trié du plus vendu au moins vendu ?

SELECT `LFA_PRO`, SUM(`LFA_QTE`) AS `Quantite_Vendue` FROM `Ligne_Facture` GROUP BY `LFA_PRO` ORDER BY `Quantite_Vendue` DESC;

LFA_PRO	Quantite_Vendue 1
F001	240
S001	218
L001	183
L002	159
L003	105
S002	55

Combien de fois ont été vendu chaque produit ?

SELECT `LFA_PRO`, COUNT(*) AS `Nb_Ventes` FROM `Ligne_Facture` GROUP BY `LFA_PRO`;

LFA_PRO	Nb_Ventes
F001	3
L001	6
L002	4
L003	2
S001	4
S002	3

Combien de produit ont été vendu au total ?

SELECT SUM(`LFA_QTE`) AS `Total_Produits_Vendus` FROM`Ligne_Facture`;

Total_Produits_Vendus
960

Combien de produit ont été vendu au total mais supérieur à 200 quantités ?

SELECT `LFA_PRO`, SUM(`LFA_QTE`) AS `Quantite_Vendue` FROM `Ligne_Facture` GROUP BY `LFA_PRO` HAVING `Quantite_Vendue` > 200;

LFA_PRO	Quantite_Vendue
F001	240
S001	218

Combien de clients habitent par ville ?

SELECT `CLI_VILLE`, COUNT(*) AS `Nb_Clients` FROM `Client` GROUP BY `CLI_VILLE`;

CLI_VILLE	Nb_Clients
Aubagne	2
Mallemort	1
Marseille	5

Combien de client habite par code postal ?

SELECT `CLI_POST`, COUNT(*) AS `Nb_Clients` FROM `Client` GROUP BY `CLI_POST`;

CLI_POST	Nb_Clients
13010	3
13400	3
13456	1
13601	1

Combien de client y-a-t-il par catégorie de client ?

SELECT `CLI_CAT`, COUNT(*) AS `Nb_Clients` FROM `Client` GROUP BY `CLI_CAT`;

CLI_CAT	Nb_Clients
BOU	2
HYP	3
PAT	1
SUP	2

Quel est le produit le plus vendu par facture ?

SELECT `LFA_FAC`, `LFA_PRO`, SUM(`LFA_QTE`) AS `Quantite_Vendue` FROM `Ligne_Facture` GROUP BY `LFA_FAC`, `LFA_PRO` ORDER BY `Quantite_Vendue` DESC LIMIT 1;

LFA_FAC	LFA_PRO	Quantite_Vendue
2	S001	150

VI- Annexe

Mdp utilisateur :

Root : SujFVcJ*M8S$sJc

Admin : 5R5sLq1u/$Rl3:

Gestion : xUk;3XoZh4*2#3

User : user